1. System Overview

GrowthPulse is a complete consumer IoT product: a sensor node a customer plugs in and pairs from their phone, a cloud pipeline, and a multi-tenant web application with real accounts, alerts, and subscription tiers. This manual documents every layer of the system, every source file and every significant function, and the reasoning behind every decision. It is an internal document and assumes engineering context.

1.1 The data path

```
Sensors (DS18B20, DHT22, capacitive soil probe)
   -> ESP32-S3 firmware (GP_Provisioning.ino)
   -> MQTT over the customer's Wi-Fi
   -> Losant device cloud (stores live state + full time-series history, runs alert workflows)
   -> Netlify serverless functions  (Losant tokens held server-side):
        device-state.js    live composite state (every refresh tick)
        device-history.js  time-series history for report graphs
        render-pdf.js      headless-Chrome vector PDF of a report
   -> React web app at growthpulsecloud.com
   -> the customer
```

1.2 The control path

```
App "Factory reset" button
   -> Netlify function device-command.js (write token, allowlisted commands)
   -> Losant command REST API
   -> MQTT command down to the unit
   -> firmware handleCommand(): wipe Wi-Fi credentials, reboot into setup mode
```

1.3 The identity chain

```
ESP32 factory MAC (burned in at chip manufacture)
   -> low 24 bits rendered as uppercase hex = pairing code (e.g. 4A7AC)
   -> shown large on the unit's OLED, and embedded in the setup hotspot name
   -> device_registry row in Supabase maps pairing code -> Losant device id
   -> customer types the code in the app -> claim validated -> device on account
```

1.4 Services used, and what each one does

Service	Used for	Link
GitHub	Source control for the web app + serverless functions; pushing to main triggers deploys	github.com/BryanPuckettGH/growthpulse-app

Service	Used for	Link
Netlify	Hosting for growthpulsecloud.com, build pipeline, serverless functions, environment variables	netlify.com
Supabase	User accounts (email/password auth) and the device_registry pairing table	supabase.com
Losant	Device cloud: MQTT broker, device state storage, email/SMS alert workflows, command delivery	losant.com
Open-Meteo	Weather forecasts and the place-name geocoder used for per-plant home locations. Free, no API key	open-meteo.com
Heltec docs	Board documentation, GPIO usage guide for the WiFi LoRa 32 V3	wiki.heltec.org
Silicon Labs	CP210x USB-to-UART driver needed to program the board	silabs.com/developer-tools/usb-to-uart-bridge-vcp-drivers
Arduino	The IDE used to build and flash the firmware	arduino.cc/en/software

Credentials policy: no usernames, passwords, keys, or tokens appear in this manual or in git. Live secrets exist in exactly three places: the gitignored `.env` on the development machine, Netlify's environment variable store, and the firmware file (which is kept out of public repositories for that reason).

---

2. Getting the Code and Setting Up

2.1 The web application

```
git clone https://github.com/BryanPuckettGH/growthpulse-app.git
cd growthpulse-app
npm install
npm run dev          # opens at http://localhost:5173
```

Requires Node.js LTS (nodejs.org). Create a `.env` file in the project root with these variable names (values come from the team's Supabase and Losant dashboards, never from this document):

```
VITE_SUPABASE_URL=
VITE_SUPABASE_ANON_KEY=
LOSANT_API_TOKEN=
LOSANT_APP_ID=
LOSANT_COMMAND_TOKEN=
```

Two things about local development:

- Variables prefixed `VITE_` are compiled into the browser bundle by Vite. Everything else is only readable by the serverless functions. That prefix is the security boundary, never put a secret behind a `VITE_` name.
- `npm run dev` runs Vite only. The serverless functions do not execute, so claimed devices sit at "waiting for first reading" locally. To exercise the full pipeline either use the deployed site or run `netlify dev` (Netlify CLI), which serves the functions too.

Deployment is automatic: any push to `main` triggers a Netlify build (`npm run build`, publish `dist/`, bundle `netlify/functions/` with esbuild, all configured in `netlify.toml`).

2.2 The firmware toolchain, from zero

Everything needed to program a unit on a fresh computer:

1. **Arduino IDE 2.x** from arduino.cc/en/software.
2. **USB driver.** The board talks USB through a Silicon Labs CP2102 bridge. Without the driver the board never shows up as a serial port. Install the CP210x Virtual COM Port driver from silabs.com/developer-tools/usb-to-uart-bridge-vcp-drivers. On macOS the port then appears as `/dev/cu.usbserial-XXXX`; on Windows as `COMn`.
3. **Board support.** Arduino IDE, Settings, Additional boards manager URLs, add the Espressif index: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`. Then Boards Manager, install **esp32 by Espressif Systems**. Select the board **"Heltec WiFi LoRa 32(V3)"**, not a generic ESP32-S3 profile, the Heltec definition carries the correct flash and pin configuration.
4. **Libraries** (Library Manager, exact names):

Library	Author	Used for
WiFiManager	tzapu	Captive-portal Wi-Fi provisioning (release 2.0.17 at time of writing)
OneWire	Paul Stoffregen	1-Wire bus for the DS18B20
DallasTemperature	Miles Burton	DS18B20 temperature conversion
DHT sensor library	Adafruit	DHT22 (installs Adafruit Unified Sensor as a dependency)
Losant Arduino MQTT Client	Losant	Device cloud connection (installs PubSubClient)
ArduinoJson	Benoit Blanchon	Telemetry payloads (v6-style StaticJsonDocument)
U8g2	oliver	The on-board OLED

1. **Open** `GP_Provisioning/GP_Provisioning.ino` (the folder name must match the file name, an Arduino requirement).

2. **Upload.** Plug in USB-C, pick the port, click Upload. If the upload hangs at "Connecting..." or dies mid-write with "the chip stopped responding," two fixes in order: set Tools, Upload Speed to **115200** (the 921600 default outruns marginal cables), and if needed force download mode by holding **PRG**, tapping **RST**, holding PRG one more second, then uploading.
3. **Serial Monitor** at **115200 baud** shows the boot banner, the pairing code, Wi-Fi progress, and a one-line telemetry summary every cycle, including the raw soil ADC value used for calibration.

2.3 What is deliberately not in GitHub

`GP_Provisioning.ino` contains the unit's Losant device id, access key, and access secret, compiled in. One demo unit makes that acceptable; publishing it would not be. The firmware therefore lives in the project folder, not the public repo. The production path (per-unit credentials written at flash time by a provisioning station) is covered in the Roadmap.

---

3. Hardware

3.1 The board

Heltec WiFi LoRa 32 V3, silkscreen HTIT-WB32LAF. Relevant capabilities:

Component	Detail
MCU	ESP32-S3 dual-core, 240 MHz, 8 MB flash
Radios	2.4 GHz Wi-Fi b/g/n, BLE, and an SX1262 LoRa transceiver (915 MHz US)
Display	0.96 inch SSD1306 OLED, 128x64, on I2C
USB	USB-C through a CP2102 UART bridge
Buttons	RST (hard reset) and PRG (user button on GPIO0)
Power	5V in via USB-C or the 5V header pin; onboard 3.3V regulator; JST connector for a LiPo battery

The SX1262 is unused by the current firmware but is the hardware foundation of the LoRaWAN roadmap: the same physical unit becomes a LoRaWAN node with a firmware variant, no new electronics.

3.2 Pin assignments, and why each one

Function	GPIO	Reasoning
DS18B20 data (1-Wire)	4	Listed as free and beginner-safe in Heltec's official GPIO guide

Function	GPIO	Reasoning
DHT22 data	5	Same list
Soil moisture analog out	2	Moved off GPIO1 after bring-up testing: GPIO1 is wired to the board's battery-voltage sense network and read a flat 0 regardless of the sensor. GPIO2 is the adjacent clean ADC1 channel
PRG button	0	The board's built-in user button; a boot-strapping pin, safe to read after boot
OLED SDA / SCL / reset	17 / 18 / 21	Fixed by board routing
OLED power rail (Vext)	36	The V3 feeds the OLED from a switched rail; the firmware must drive GPIO36 LOW to power it. Forgetting this is the classic blank-screen failure

Analog constraints worth memorizing: analog inputs must use ADC1 channels (GPIO1 through 10) because ADC2 is owned by the Wi-Fi radio while it runs. Resolution is configured to 12 bits, so raw readings span 0 to 4095.

3.3 Sensor electronics

DS18B20, soil temperature

A digital 1-Wire device in a waterproof probe. Wiring: red to 3V3, black to GND, yellow (data) to pin 4, **plus a 4.7 kilo-ohm resistor from the data line to 3V3**. The resistor is not optional and the reason is the bus design: on 1-Wire, the master and every sensor only ever pull the line LOW. Nothing on the bus can drive it high. The pull-up resistor is the only thing that returns the line to 3.3V between pulses. Without it the line never rises, communication is impossible, and the Dallas library returns its "no device" sentinel of -127 C, which converts to the famous **-196.6 F**. The app now recognizes that exact value (section on disconnected-sensor detection) and turns it into a "probe not detected" message naming the resistor.

Each DS18B20 also carries a unique 64-bit serial number whose first byte (the family code) is 0x28. During bring-up we used a 1-Wire scanner sketch that searches the bus and prints these addresses; a found address proves the wiring end-to-end even if temperature conversion were broken, and a storm of random non-0x28 addresses indicates electrical noise on a floating line rather than real devices.

DHT22, air temperature and humidity

A single-wire (not 1-Wire) digital sensor: VCC to 3V3, data to pin 5, 10 kilo-ohm pull-up from data to VCC (the common 3-pin breakout module has this resistor built in; a bare 4-pin part needs it added). When the sensor is absent the library returns NaN; ArduinoJson serializes NaN as JSON null; the connector passes null through; and the app reads null as "not connected." The failure mode was designed into the pipeline rather than hidden.

Capacitive soil moisture probe

The flat-blade capacitive probe (v1.2 style): VCC, GND, AO (analog out). Two hard-won findings:

1. **It must be powered from 5V, not 3.3V.** These modules build their oscillator around an NE555 timer, and the NE555 does not run below roughly 4 volts. At 3.3V the power LED lights (LEDs do not care) but the oscillator is dead and AO sits near 0.1V, which reads as raw ~120 and maps to a frozen "100 percent" moisture. Vendors list the modules as 3.3 to 5.5V; the ones with NE555 silicon are 5V parts in practice. At 5V supply the analog output tops out near 3V, which is inside the ESP32 ADC's safe range, so no level shifting or divider is required.
2. **Raw, not percent, is the calibration currency.** The firmware reports both. Calibration procedure: read `raw=` from the Serial line with the probe in dry air (that number becomes `dryValue`, typically near 3600) and submerged to its line in water (`wetValue`, typically near 1300), then update the two constants. Note the inversion: wetter soil means lower raw counts, which is inherent to how the capacitance loads the oscillator.

Wiring summary

Sensor	VCC	GND	Signal	Extra
DS18B20	3V3	GND	pin 4	4.7k pull-up, data to 3V3, required
DHT22	3V3	GND	pin 5	10k pull-up (built into 3-pin modules)
Soil moisture	**5V**	GND	pin 2	none; powered from 5V on purpose

A printable wiring diagram and a breadboard-level guide live in `docs/GrowthPulse Wiring Diagram.pdf` and `docs/GrowthPulse Wiring Guide.md`.

3.4 Power

The board takes 5V in (USB-C from any phone charger, or a bench supply at 5.0V with roughly a 1A current limit feeding the 5V and GND header pins). Measured behavior: about 150 mA idle, with 300 to 500 mA bursts during Wi-Fi transmit; brownout resets during flashing or Wi-Fi joins are a cable or supply problem, not a firmware problem. Sensors draw single-digit milliamps and are powered from the board's regulated rails as in the table above.

Battery readiness. Moving the soil probe off GPIO1 had a second benefit: GPIO1 is the battery-sense input, and it is now free. Battery support is therefore a contained firmware task: enable the sense divider, read VBAT, convert to percent, and add `batteryPct` and `charging` to the telemetry JSON. The cloud connector already forwards those two fields and the entire app UI for them (power badges, color thresholds, low-battery insights) is built and waiting.

4. Firmware, Section by Section

The firmware is one annotated file, `GP_Provisioning.ino`, about 350 lines. One file is a deliberate choice at this scale: it can be reviewed top to bottom, pasted whole into the IDE, and handed to a teammate without explaining a build system. This chapter walks every section in file order and explains each decision.

4.1 Includes and pin definitions

```
#include <WiFi.h>
#include <WiFiManager.h>     // tzapu: captive portal provisioning
#include <OneWire.h>
#include <DallasTemperature.h>
#include <DHT.h>
#include <Losant.h>          // MQTT device client
#include <ArduinoJson.h>
#include <Wire.h>
#include <U8g2lib.h>         // OLED

#define ONE_WIRE_BUS 4       // DS18B20
#define DHTPIN 5             // DHT22
#define DHTTYPE DHT22
#define SOIL_PIN 2           // moved off GPIO1 (battery-sense conflict)
#define RESET_BTN 0          // PRG button

#define VEXT_PIN 36          // LOW powers the OLED rail on the V3
#define OLED_SDA 17
#define OLED_SCL 18
#define OLED_RST 21
U8G2_SSD1306_128X64_NONAME_F_HW_I2C oled(U8G2_R0, OLED_RST, OLED_SCL, OLED_SDA);
```

The U8g2 constructor takes rotation, then reset, clock, data. Passing the V3's exact pins matters; the generic constructor assumes default I2C pins and produces a blank screen.

4.2 Calibration constants and unit identity

```
int dryValue = 3600;   // raw ADC with the probe in dry air
int wetValue = 1300;   // raw ADC submerged in water

const char* LOSANT_DEVICE_ID    = "...";  // this unit's cloud identity
const char* LOSANT_ACCESS_KEY   = "...";  // values not reproduced here;
const char* LOSANT_ACCESS_SECRET= "...";  // they live only in the firmware file
```

`dryValue` and `wetValue` are intentionally plain globals near the top: they are the two numbers a technician edits after the calibration procedure (section 3.3). The Losant triplet is this single demo unit's identity, compiled in; the production replacement is per-unit credentials written at flash time.

4.3 The branded captive portal markup

Two raw-string constants are injected into WiFiManager's portal pages:

- `GP_HEAD` : a `<style>` block restyling the stock portal with brand colors (green buttons, brand typography, soft backgrounds). Injected into every portal page via `setCustomHeadElement`.
- `GP_BRANDING` : a block of HTML carrying the GrowthPulse logo as inline SVG, the wordmark, the SMART PLANT MONITORING tagline, and a "thanks for your purchase" welcome. It rides in as a `WiFiManagerParameter`, which WiFiManager renders at the top of the configuration page.

Why inline SVG: the customer's phone is connected to the device's hotspot with no internet, so the page cannot reference any external image. Everything the portal shows must be served from the chip itself.

4.4 Identity helpers

```
String pairCode() {
  char b[8];
  snprintf(b, sizeof(b), "%X", (uint32_t)(ESP.getEfuseMac() & 0xFFFFFF));
  return String(b);
}
String apSsid() {
  char b[24];
  snprintf(b, sizeof(b), "GrowthPulse-%06X", (uint32_t)(ESP.getEfuseMac() & 0xFFFFFF));
  return String(b);
}
```

`ESP.getEfuseMac()` returns the factory-burned MAC, unique per chip, free, and permanent. The low 24 bits are formatted as uppercase hex: `%X` (no leading zeros) for the human pairing code, `%06X` (zero-padded) inside the hotspot SSID so the network name has a consistent shape. The pairing code is the same value the registry stores and the app validates, one identity, three surfaces.

4.5 The OLED stack

```
void oledInit() {
  pinMode(VEXT_PIN, OUTPUT);
  digitalWrite(VEXT_PIN, LOW);   // power the display rail
  delay(80);                     // let the rail settle before init
  oled.begin();
  oled.setContrast(255);
}
```

`drawBigCentered()` implements adaptive type for the pairing code: it measures the string in FreeUniversal Bold 30, then 25, then 20, and draws with the largest font that fits in 124 px, horizontally centered. The pairing code is the one thing a customer must read off the hardware, so it gets the biggest legible rendering regardless of whether a given chip's code is 5 or 6 characters.

Four screens, all built the same way (clear buffer, set font, draw strings, send buffer):

Screen	When	Content
`oledMessage(l1, l2)`	transient states	bold headline plus detail line ("Starting up", "Connecting to your Wi-Fi...", "Clearing Wi-Fi")
`oledSetup()`	the setup hotspot is open	"Setup - join Wi-Fi:", the SSID bold and centered, "then open the app"
`oledStatus(online)`	steady state	"PAIR CODE" header, ON or .. connection chip, the code huge
(via `onSetupPortal`)	WiFiManager AP callback	flips to `oledSetup` at the exact moment the portal opens

The status screen never hides the code after pairing, on purpose: re-pairing, adding to a second account someday, and resale all need it, and it doubles as the unit's serial number.

4.6 connectWiFi(), line by line

```
void connectWiFi() {
  WiFiManager wm;
  // (setPreloadWiFiScan needs a newer WiFiManager release; skipped here.)
  wm.setWiFiAutoReconnect(false);  // stop background retries from yanking
                                   // the radio off the hotspot channel
  WiFi.setSleep(false);            // keep the AP responsive to the phone
  wm.setCustomHeadElement(GP_HEAD);
  WiFiManagerParameter gpBranding(GP_BRANDING);
  wm.addParameter(&gpBranding);
  wm.setAPCallback(onSetupPortal);
  wm.setConfigPortalTimeout(600);  // 10 minutes

  String ap = apSsid();
  oledMessage("Connecting to", "your Wi-Fi...");
  if (!wm.autoConnect(ap.c_str())) {
    oledMessage("Setup timed out", "restarting...");
    delay(1500);
    ESP.restart();
  }
  oledStatus(false);
}
```

Reasoning per line:

- **autoConnect** does the whole provisioning policy: try saved credentials; if none or they fail, open the branded hotspot and serve the portal; return true once associated. One call, no custom state machine.

- **setWiFiAutoReconnect(false)** and **WiFi.setSleep(false)** fix a real observed failure: while the portal is open the chip runs AP+STA simultaneously, and both background STA retry scans and modem power-save pull the radio off the AP channel. The symptom is the customer's phone hanging on "connecting..." to the hotspot for a minute or more. With these two lines the join completes in seconds.
- **setConfigPortalTimeout(600)**: the 180-second default expired mid-demo while a phone was slow to pop the portal, restarting the AP underneath the user. Ten minutes gives a human-paced setup window; on timeout the device restarts and reopens setup rather than wedging.
- **setPreloadWiFiScan(true)** would move the network scan before the AP opens (another join-speed win) but the setter only exists in WiFiManager's unreleased development code; release 2.0.17 compiles without it, so it is deliberately omitted with a comment.
- The portal also responds at **192.168.4.1** directly, the documented fallback when a phone suppresses captive-portal detection.

4.7 Cloud command handler

```
void handleCommand(LosantCommand *command) {
  if (strcmp(command->name, "factoryReset") == 0) {
    oledMessage("Reset by owner", "clearing Wi-Fi...");
    delay(800);
    WiFiManager wm;
    wm.resetSettings();   // erase stored Wi-Fi credentials
    delay(400);
    ESP.restart();        // boots into the setup hotspot
  }
}
```

Registered once in `setup()` with `device.onCommand(&handleCommand)`. This is the device half of the app's resale flow: the owner taps Factory reset, the cloud delivers `factoryReset` over the existing MQTT connection, and the unit physically prepares itself for the next owner, on its screen, in front of you. Commands reach online devices only; the PRG hold is the offline fallback and the app's confirmation dialog says exactly that. The handler ignores every other command name, an allowlist at the firmware level matching the allowlist in the backend function.

4.8 connectLosant()

Connects the MQTT client with the device id, access key, and secret, blocking with a dot-progress loop until connected, with OLED messaging. Losant access keys are device credentials (MQTT) and are distinct from API tokens (REST, used by the backend); the two never mix.

4.9 setup()

Order matters and each step has a reason:

1. `Serial.begin(115200)`, `analogReadResolution(12)` (12-bit ADC, 0-4095, matching the calibration constants), PRG configured `INPUT_PULLUP`, a 1-second settle delay.
2. `oledInit()` then a "Starting up" screen, the customer sees life within a second of plugging in.
3. **Boot-time reset check:** if PRG is held LOW right now, wipe Wi-Fi credentials and continue, which lands in the setup portal. This is reset path one of three.
4. Serial banner with the pairing code, for bench work without squinting at the OLED.
5. `ds18b20.begin()`, `dht.begin()`, then `device.onCommand(&handleCommand)` before any connection exists, so no command can ever arrive unhandled.
6. `connectWiFi()` then `connectLosant()`.

4.10 readSensors()

```
void readSensors() {
  ds18b20.requestTemperatures();
  soilTemperatureF = ds18b20.getTempFByIndex(0);   // -196.6F when absent

  airTemperatureF = dht.readTemperature(true);      // NaN when absent
  airHumidity = dht.readHumidity();

  long sum = 0;
  for (int i = 0; i < 10; i++) { sum += analogRead(SOIL_PIN); delay(10); }
  soilRaw = sum / 10;                                // 10-sample average
  soilMoisturePercent = constrain(map(soilRaw, dryValue, wetValue, 0, 100), 0, 100);
}
```

Decisions: sensor failure values pass through untouched (the app turns them into honest UI rather than the firmware guessing); the soil ADC is averaged over 10 samples 10 ms apart because single ESP32 ADC reads jitter by several counts; `map` runs dry-to-wet which inverts the scale correctly since wet soil reads lower; `constrain` clamps readings outside the calibration window instead of reporting 112 percent.

4.11 sendTelemetry()

Builds a `StaticJsonDocument<256>` with the five values (`soilTemperatureF`, `airTemperatureF`, `airHumidity`, `soilRaw`, `soilMoisturePercent`) and publishes with `device.sendState`. `soilRaw` ships on purpose: recalibration and the app's floating-probe heuristic both need the raw signal. Serial mirrors a single line per cycle (`Sent moisture=42% raw=2410 airTemp=78.8F`), which is the field calibration tool, no extra software needed.

4.12 maybeResetWifi(), reset path two

```
void maybeResetWifi() {
  static unsigned long pressStart = 0;
  if (digitalRead(RESET_BTN) == LOW) {
    if (pressStart == 0) pressStart = millis();
    else if (millis() - pressStart >= 3000) {
      oledMessage("Clearing Wi-Fi", "reopening setup");
      delay(600);
      WiFiManager wm; wm.resetSettings(); delay(400);
      ESP.restart();
    }
  } else {
    pressStart = 0;   // released early: reset the timer
  }
}
```

Hold PRG for 3 seconds at any time while running and the unit wipes Wi-Fi and reboots into setup. This replaced the original hold-PRG-during-RST dance, which demanded sub-second timing customers reliably missed. Design choice: a polled timestamp rather than interrupts, trivially safe alongside two radio stacks, and tolerant of the loop's sensor-read gaps because `pressStart` only clears when the button is actually seen released.

4.13 loop()

```
void loop() {
  if (WiFi.status() != WL_CONNECTED) connectWiFi();
  if (!device.connected())          connectLosant();

  readSensors();
  sendTelemetry();
  device.loop();                                    // service MQTT (commands arrive here)
  oledStatus(WiFi.status() == WL_CONNECTED && device.connected());

  unsigned long waitStart = millis();               // 3s between sends, but
  while (millis() - waitStart < 3000) {             // sampled every 20ms so a
    maybeResetWifi();                               // 3s PRG hold is never missed
    delay(20);
  }
}
```

Self-healing first (reconnect either layer that dropped), then read, send, service, display, and a button-aware wait instead of a deaf `delay(3000)`. The 3-second cadence is human-real-time for demos while staying far inside the Losant free tier; battery units will stretch it dramatically.

4.14 The diagnostic sketches

Two single-purpose sketches exist for bench debugging and are flashed temporarily over the main firmware:

- **ADC scanner**: prints GPIO 1, 2, 3, 6, 7 raw values side by side twice a second. Used with a jumper test (touch 3V3 to a pin, expect ~4095; touch GND, expect ~0) to prove an ADC pin and to find which pin a wire actually lands on. This is how the GPIO1 battery-sense conflict was isolated.
- **1-Wire scanner**: searches pins 4, 6, 7 for 1-Wire devices and prints their 64-bit addresses. A real DS18B20 announces with family code 0x28; "nothing" on all pins means the probe never reaches the bus (power, contact, or pull-up); a burst of random addresses is line noise being misparsed, which actually proves the pin and code are alive.

Re-flashing the main firmware afterward restores normal operation; Wi-Fi credentials survive in flash across re-flashes unless explicitly reset.

5. The Device Cloud (Losant)

Losant (losant.com) is the device-facing cloud: it terminates the MQTT connection from every unit, stores the latest state per device, runs the alert workflows, and delivers commands back down.

5.1 Device and attributes

One application contains one device per physical unit. The demo unit is the standalone device "Greenhouse Node" (device id `6a1fb486df527c8bf8d3324b`, not a secret, it appears in claim plumbing). Its attributes mirror the telemetry JSON exactly:

Attribute	Type	Notes
airTemperatureF	number	DHT22
airHumidity	number	DHT22
soilTemperatureF	number	DS18B20, carries the -196.6 sentinel when unplugged
soilRaw	number	raw soil ADC, kept for calibration and diagnostics
soilMoisturePercent	number	the derived percentage
batteryPct, charging	number, boolean	reserved for battery firmware; connector already forwards them

5.2 Credentials model: keys vs tokens

Two different credential systems, never interchangeable:

- **Access keys** (key + secret pair) authenticate **devices** over MQTT. The unit's key/secret is compiled into the firmware. Scope: that application's devices.
- **API tokens** authenticate **applications** over REST. Two exist, on purpose:
- a **read-only token** (scope `all.Application.read`) used by `device-state.js` to read composite state. If it ever leaked, it can only read.
- a **full-access token** used by `device-command.js` to send commands. It exists separately so the hot, frequently-called read path runs on least privilege.

5.3 The wire protocol

- Telemetry up: MQTT publish to `losant/{deviceId}/state` (handled inside the Losant Arduino client by `device.sendState`).
- Commands down: MQTT message on `losant/{deviceId}/command`, surfaced by the client as the `onCommand` callback. Delivery requires the device to be connected at that moment; there is no offline queue, which is why the app's factory-reset dialog documents the PRG fallback.
- State read by the backend: REST `GET /applications/{appId}/devices/{deviceId}/compositeState`, returning per-attribute `{ value, time }` pairs.

5.4 Alert workflows (email and SMS)

Built in Losant's visual workflow editor:

```
Device: State trigger  ->  Conditional  ->  Email node (and/or SMS)
```

- The trigger fires on every state report from the device.
- The conditional holds the rule, for example `{{ data.soilMoisturePercent }} < 25`.
- The true branch wires to the Email node (the false branch goes nowhere). A wiring mistake to watch for, learned the hard way: connect the conditional's true output to the email node's input; routing the email node backwards into the conditional silently does nothing.
- **Rate limit:** Losant email sends are limited to one per minute. During testing, two qualifying readings sent seconds apart produced one email, which looked like a failure and was not. SMS goes through the corresponding SMS node where enabled.

These cloud workflows are what make alerts real-world: they fire even when nobody has the app open. The app's in-app alarm system (chapter 8) is a separate, complementary layer.

5.5 The simulator

`growthpulse-simulator/simulator.mjs` is a Node script that connects with the same MQTT credentials and publishes realistic randomized telemetry. It built the entire cloud pipeline before hardware existed and remains the demo tool when the physical unit is elsewhere. Operational rule: never run it at the same time as the real board against the same device id, the two sources interleave into nonsense.

---

6. Accounts and the Registry (Supabase)

Supabase (supabase.com) provides two things: authentication and the pairing registry.

6.1 Authentication

Email/password auth via `@supabase/supabase-js`. Signup attaches profile metadata so the app can address people properly:

```
supabase.auth.signUp({ email, password,
  options: { data: { first_name, last_name, grower_type } } })
```

The metadata lands in `user.user_metadata` and flows into the UI (greeting name, avatar initial, grower-type badge in Settings). Sessions persist in browser storage and are observed with `onAuthStateChange`, so a refresh never logs anyone out.

6.2 The device registry

The full schema, as deployed (also in `docs/growthpulse-supabase-schema.sql`):

```
create table if not exists device_registry (
  claim_code         text primary key,
  losant_device_id   text not null,
  created_at         timestamptz default now()
);

alter table device_registry enable row level security;

create policy "authenticated can read registry"
  on device_registry for select
  to authenticated
  using (true);

insert into device_registry (claim_code, losant_device_id)
values ('4A7AC', '6a1fb486df527c8bf8d3324b')
on conflict (claim_code) do nothing;
```

Reading it as a product artifact: this table **is the factory database**. Every manufactured unit gets one row at provisioning time, mapping the code shown on its screen to its cloud identity. Row-level security is the whole access model: authenticated users may look codes up (that is what claiming is), and nobody can write through the public anon key because no insert/update policy exists. Writes happen only through the service role in the dashboard or, later, a provisioning station.

Why the anon key is allowed to be public: it is designed that way by Supabase; RLS policies are the enforcement layer. The key ships in the browser bundle (it is a `VITE_` variable) and grants exactly what the policies say and nothing more.

6.3 The devices table: cloud-authoritative ownership

The registry says which codes are valid; the **devices** table says who owns what. It makes claims cloud-authoritative (a claim follows the account to any browser instead of living in one browser's localStorage), enforces one-owner-per-unit, and gives the command endpoint something to authorize against. Schema as deployed (`docs/growthpulse-devices-schema.sql`):

```
create table if not exists devices (
  id                uuid primary key default gen_random_uuid(),
  user_id           uuid not null default auth.uid() references auth.users(id) on delete cascade,
  claim_code        text not null unique,           -- one account per unit
  losant_device_id  text not null,
  name              text, location text, geo jsonb, grp text,
  transport         text default 'wifi', plant text default 'generic',
  irrigation        jsonb,
  created_at        timestamptz default now()
);

alter table devices enable row level security;

create policy "owner reads"   on devices for select to authenticated using (auth.uid() = user_id);
create policy "owner inserts" on devices for insert to authenticated with check (auth.uid() = user_
create policy "owner updates" on devices for update to authenticated using (auth.uid() = user_id);
create policy "owner deletes" on devices for delete to authenticated using (auth.uid() = user_id);
```

Notes that matter:

- **`claim_code` is `UNIQUE`**, so a second account claiming the same code hits a `23505` violation, which the store surfaces as "already claimed by another account." That is claim exclusivity in one constraint.
- **`grp`** is the column name (`group` is a SQL reserved word); the app maps `r.grp <-> device.group` .
- **`geo` and `irrigation` are `jsonb`**, so the plant's home coordinates and the full watering config (including `rainDelay`) ride in the row.
- **Per-user RLS on all four verbs** means the browser talks to the table directly with the anon key and can still only ever see or change its own rows; there is no trusted server in this path, the database is the guard.

- `created_at` is read into the device as `claimedAt`, which drives the report's "Everything" range and the activity timeline.

A one-time migration lifts pre-ownership local claims into the table on first load: for each locally-stored claim it recovers the `claim_code` from the registry, inserts the row, and remaps that browser's journals and device-scoped alarms onto the new cloud id.

---

7. The Backend (Netlify Functions)

Netlify (netlify.com) hosts the static app and two serverless functions from `netlify/functions/`, bundled with esbuild per `netlify.toml`:

```
[build]
  command = "npm run build"
  publish = "dist"
[functions]
  directory = "netlify/functions"
  node_bundler = "esbuild"
```

The functions exist for exactly one reason: **no Losant token may ever reach a browser.** The app calls same-origin endpoints; the tokens live in Netlify environment variables readable only by function code.

7.1 device-state.js, the read path

```
export const handler = async (event) => {
  const token = process.env.LOSANT_API_TOKEN;       // read-only token
  const appId = process.env.LOSANT_APP_ID;
  const deviceId = event.queryStringParameters?.deviceId;

  if (!token || !appId) return { statusCode: 500, body: '{"error":"Server not configured"}' };
  if (!deviceId)        return { statusCode: 400, body: '{"error":"deviceId required"}' };

  const res = await fetch(
    `https://api.losant.com/applications/${appId}/devices/${deviceId}/compositeState`,
    { headers: { Authorization: `Bearer ${token}` } });
  if (!res.ok) return { statusCode: res.status, body: '{"error":"Device not found"}' };

  const s = await res.json();
  const num  = (k) => (s[k] && typeof s[k].value === 'number'  ? s[k].value : null);
  const bool = (k) => (s[k] && typeof s[k].value === 'boolean' ? s[k].value : null);
  const reading = {
    airTemperatureF: num('airTemperatureF'),
    airHumidity: num('airHumidity'),
    soilTemperatureF: num('soilTemperatureF'),
    soilRaw: num('soilRaw'),
    soilMoisturePercent: num('soilMoisturePercent'),
```

```
    batteryPct: num('batteryPct'),
    charging: bool('charging'),
    time: s.soilMoisturePercent ? new Date(s.soilMoisturePercent.time).getTime() : Date.now(),
  };
  return { statusCode: 200,
    headers: { 'Content-Type': 'application/json', 'Cache-Control': 'no-store' },
    body: JSON.stringify(reading) };
};
```

Design notes: the `num` / `bool` guards convert anything missing or mistyped into `null`, which is the app's universal "no data" signal, the same null that means an unplugged DHT22. `Cache-Control: no-store` because a stale reading is worse than an extra request for live monitoring. The timestamp rides on the moisture attribute's report time so the app can show data age honestly.

7.2 device-command.js, the write path

```
export const handler = async (event) => {
  if (event.httpMethod !== 'POST') return { statusCode: 405, ... };
  const token = process.env.LOSANT_COMMAND_TOKEN;    // separate write token
  const { deviceId, name = 'factoryReset' } = event.queryStringParameters || {};
  if (name !== 'factoryReset') return { statusCode: 400, ... };  // allowlist

  const res = await fetch(
    `https://api.losant.com/applications/${appId}/devices/${deviceId}/command`,
    { method: 'POST',
      headers: { Authorization: `Bearer ${token}`, 'Content-Type': 'application/json' },
      body: JSON.stringify({ name, payload: {} }) });
  ...
};
```

Four defenses now: POST only, a one-entry command allowlist, the dedicated write token, **and an ownership check** (the former gap, now closed by §6.3). Before sending anything, the function takes the caller's forwarded Supabase session and queries the `devices` table for a row matching this `losant_device_id`; RLS returns only rows the caller owns, so zero rows means 403 and no auth header means 401:

```
const callerAuth = event.headers.authorization;          // the user's Supabase JWT
if (!callerAuth) return { statusCode: 401, ... };         // must be signed in
const own = await fetch(
  `${supaUrl}/rest/v1/devices?losant_device_id=eq.${deviceId}&select=id`,
  { headers: { apikey: supaKey, Authorization: callerAuth } });
const rows = own.ok ? await own.json() : [];
if (rows.length === 0) return { statusCode: 403, ... };   // not your device
// ...only now send the Losant command
```

The browser supplies the header from its live session (`factoryResetDevice` calls `supabase.auth.getSession()` and sends `Authorization: Bearer <access_token>`). The database, via RLS, is the authority; the function just relays its verdict.

7.3 Environment variables, the complete set

Variable	Visible to	Purpose
VITE_SUPABASE_URL	browser bundle	Supabase project URL
VITE_SUPABASE_ANON_KEY	browser bundle	public anon key; RLS enforces access
LOSANT_APP_ID	functions only	Losant application id
LOSANT_API_TOKEN	functions only, marked secret	read-only state token (also serves history queries)
LOSANT_COMMAND_TOKEN	functions only, marked secret	command-capable token

Set in Netlify under Site configuration, Environment variables; locally in the gitignored `.env`. The `VITE_` prefix is the line between public and private, enforced by Vite at build time. The two newer functions add **no new secrets**: `device-history` reuses the read-only `LOSANT_API_TOKEN`, and `render-pdf` reuses the Supabase pair to authenticate the caller and needs no Losant access at all.

7.4 device-history.js, the history path

Backs the report graphs. The app only holds a 60-reading in-memory ring, so anything longer than the last few minutes comes from Losant's time-series store. `GET ?deviceId&start&end` (ms epoch) issues a `POST /data/time-series-query` with the read-only token:

```
const MAX_POINTS = 240, MINUTE = 60000;
const resolution = Math.max(MINUTE, Math.ceil((end - start) / MAX_POINTS / MINUTE) * MINUTE);
// body: { start, end, resolution, aggregation:'MEAN',
//         attributes:[air/soil...], deviceIds:[deviceId], order:'asc' }
```

The resolution auto-scales so a report never carries more than ~240 chart points regardless of span (a year collapses into ~1.5-day buckets; a day stays at minute detail). The same disconnected-sensor sentinels handled everywhere else are cleaned to `null` here too (soil temp < -100, humidity <= 0, the DHT22 "temp 0 with no humidity" case, soil raw < 300), so a dead probe leaves an honest gap in the graph instead of dragging the averages. Empty lead/tail buckets (an "Everything" range that starts before the unit existed) are trimmed; interior gaps are kept so offline stretches stay visible.

7.5 render-pdf.js, the server-side report renderer

Produces a true vector PDF, crisp selectable text, from a finished report's HTML using headless Chrome. `POST { html, filename }`:

```
import chromium from '@sparticuz/chromium';   // Lambda-sized Chromium binary
import puppeteer from 'puppeteer-core';
```

```
// 1) auth-gate: verify the caller's Supabase token (GET /auth/v1/user) -> 401 if invalid
// 2) launch chromium, page.setContent(html, {waitUntil:'networkidle0'})
// 3) page.pdf({ format:'letter', printBackground:true, margin:{...} })
// 4) return base64 PDF with Content-Disposition: attachment
```

It is **auth-gated** (a valid session is required) so it can't be abused as an open HTML-to-PDF service; it does not trust the HTML to name a device, it only confirms the caller is a signed-in user. `netlify.toml` keeps the native binary out of the esbuild bundle and ships it alongside the function:

```
[functions."render-pdf"]
  external_node_modules = ["@sparticuz/chromium", "puppeteer-core"]
  included_files = ["node_modules/@sparticuz/chromium/**"]
```

This is the **preferred** download path. The client tries it first and falls back to in-browser rendering (see §9.10) if the caller isn't signed in (demo mode), the function is slow/unavailable, or it returns anything but a PDF, so the download always succeeds.

---

8. Identity and Claiming, End to End

The complete journey of one unit from factory to a customer's dashboard:

1. **Factory (today: the bench).** Flash the firmware; read the pairing code from the OLED or Serial; create the Losant device and access key; insert the registry row mapping code to device id. For the demo unit: `4A7AC -> 6a1fb486df527c8bf8d3324b`.
2. **Customer Wi-Fi setup.** Power on; the unit opens `GrowthPulse-XXXXXX`; the customer joins from a phone and submits home Wi-Fi on the branded portal; the unit stores credentials, connects, and starts streaming. No app, account, or cloud knowledge involved in this step.
3. **Claim.** In the app: Connect a device, choose Wi-Fi or LoRaWAN gateway, name the plant, type its home city or ZIP, and enter the code from the screen. `claimDevice` uppercases and trims the code, looks it up in `device_registry`, **rejects unknown codes** with a human message, geocodes the home location (failure is non-fatal, weather just falls back), and adds the device with its `losantDeviceId` attached.
4. **Live.** The store's polling loop begins fetching `device-state` for the new device. It renders as "waiting for the first reading" with dashes until real data arrives, never invented numbers, then flips online.

Why a typed code instead of fully automatic binding: during step 2 the phone is on the unit's hotspot with **no internet**, so it cannot reach the account backend to bind silently. A short displayed code typed once is the standard consumer-IoT answer (and what shipped); the zero-typing upgrade, where the portal carries a one-time account token that the unit POSTs to a bind endpoint on first connect, is specced in the Roadmap.

Why validation matters: an early build accepted any string as a pairing code and manufactured a fake-looking device for it. The registry lookup closed that, a typo now produces "That pairing code wasn't recognized" instead of a ghost plant.

9. The Web Application, File by File

Stack: React 18 + Vite, recharts for charts, lucide-react for icons, no router (a five-tab state machine), one hand-written CSS design system. This chapter covers every source file.

9.1 Entry: index.html and main.jsx

`index.html` carries the PWA surface: `viewport-fit=cover` (content may extend into iPhone safe areas, which the CSS then respects), `theme-color`, the SVG favicon, `manifest.webmanifest`, `apple-touch-icon.png`, and the three Apple metas (`apple-mobile-web-app-capable`, status-bar style, app title) that make Add to Home Screen produce a real full-screen app. `main.jsx` is the standard five-line React root with StrictMode.

9.2 supabaseClient.js

Creates the client from the two `VITE_` variables and exports `supabaseConfigured` so the app can render a friendly "accounts service not connected" notice instead of crashing when env vars are missing (fresh clones, CI).

9.3 auth/AuthProvider.jsx

A context with `{ user, isDemo, authReady, login, signup, logout, startDemo, configured }`.

- On mount: `getSession()` restores any existing session, then `onAuthStateChange` keeps `user` current; `authReady` gates the first paint so the app never flashes the login for an already-signed-in user.
- `signup(email, password, meta)` forwards profile metadata (first/last name, grower type) into Supabase user metadata.
- **Demo mode** is a parallel fake session: `startDemo()` sets a flag and the provider serves `DEMO_USER { id: 'demo' }` as the effective user. Demo is therefore just another user id to the rest of the app, which is the trick that keeps demo data and real data perfectly separated (see storage namespacing).

9.4 App.jsx: Gate and Shell

`Gate` is the top-level decision: unconfigured notice, loading, `<Login/>`, or `<AppProvider key={user.id}><Shell/></AppProvider>`. The `key={user.id}` is load-bearing:

changing user identity remounts the entire store so no state can leak between accounts or between demo and real.

`Shell` owns: the tab state machine (`live | history | alarms | devices | settings`), the theme hook (`data-theme` attribute driven by the setting, with an `auto` mode that tracks the OS via `matchMedia`), the app bar (selected device name and location, the share-report button, the avatar), the bottom tab bar (which CSS transforms into a desktop sidebar at 860px), the alarm badge, and the toast system. Toasts work by diffing: each render computes the set of currently tripped alert keys (`deviceId:ruleId`); any key that exists now but not in the previous set raises a 4-second toast naming the device and rule. Diffing keys rather than counts means three simultaneous trips do not collapse into one anonymous notification.

When `devices.length === 0` , Shell returns `<Onboarding/>` instead of the tab UI, the claim-first welcome screen. Onboarding renders inside the full-page centered wrapper, not the shell, because the desktop shell grid once auto-placed it into the 240px sidebar column (a real shipped bug, documented in the runbook).

9.5 store/AppContext.jsx, the store

One context provides everything; there is no Redux because one provider with `useCallback` actions is sufficient at this scale.

Persistence and isolation. `load` / `save` wrap localStorage with the key scheme `growthpulse:<userId>:<name>` . Combined with the provider remount on user change, every account (and the demo) has fully isolated devices, settings, alarms, journals, gateways, and tier.

Device model.

```
{ id, name, location, geo {lat, lon, label}, group, transport: 'wifi'|'lorawan',
  plant, irrigation {mode, targetMoisture, durationSec, enabled, pausedUntil, rainDelay},
  losantDeviceId?,           // present = a real claimed unit
  claimedAt,                 // ms epoch; when the plant joined the account
  reading, history[<=60], hasData, online, lastSeen, pumpRunning }
```

`buildDevice` normalizes any persisted shape: legacy `ethernet` transports coerce to `wifi` , defaults fill missing fields, and, critically, **claimed devices initialize with a null reading, `hasData:false` , `online:false` .** They render as "Waiting for the first reading" with dashes. Only the arrival of real cloud data flips them live. Demo devices initialize from `seedReading()` instead. This split exists because an early bug let fake-looking data appear for devices that had never reported.

Cloud-authoritative devices. For real accounts, devices are not seeded from localStorage; the store initializes to `[]` and a `devicesReady` flag gates the UI while it loads `supabase.from('devices').select('*').order('created_at')` . `rowToDevice(r)` maps a table row into the device shape (`r.grp -> group` , `r.created_at -> claimedAt` , the row `id` as the device id). `syncDevice(id, patch)` pushes changed fields back (name, location, geo, grp, transport, plant,

irrigation). Demo mode is the only path that still persists to localStorage. The one-time legacy-claim migration described in §6.3 runs here.

The live loop. One `setInterval` at the user's refresh rate. Each tick: claimed devices are fetched in parallel from `device-state` (errors keep the previous reading; a `devicesRef` lets the closure read current state without re-arming the interval), and demo devices advance via `nextReading`, a bounded random walk. Updates append to a 60-entry rolling history that powers trends and the metric detail charts. Staleness flips `online:false` after 45 s of silence on Wi-Fi (15 min on LoRaWAN, which reports far less often by design).

Weather, pinned to the plant, location optional. The selected device's stored `geo` drives an Open-Meteo forecast call (current temperature and weather code, daily rain probability, UV, high; unit-aware). If the device has **no** `geo`, the store sets `weather = { needsLocation: true }` and stops, it never reads browser/GPS location and never invents a default city. The card then invites the user to add a location. This is deliberate: customers who don't want to share where they live are not forced to, and the only feature that actually needs a location (rain delay) is the one that asks for it, with the reason shown. The effect re-runs when units or the pinned coordinates change. It is still the answer to the vacation problem: the forecast describes where the plant lives, not where the owner is standing.

Actions, each one line of intent:

- `addDevice` (demo only): adds a simulated device.
- `claimDevice(code, name, place, transport)`: registry validation, optional geocode, then for real accounts an `insert` into the `devices` table (a `23505` unique-violation becomes "already claimed by another account"); demo adds locally. Returns an error string or null, which the claim sheet renders.
- `setDevicePlant`, `updateDevice` (general patch: rename, location with re-geocode, geo, group, transport, all mirrored to the cloud via `syncDevice`).
- `setIrrigation(id, patch)`: merges the watering config (including `rainDelay`) and syncs the whole `irrigation` object to the row.
- `removeDevice(id)`: removes the device, **purges** its journal entries and device-scoped alarm rules, and `delete`s the cloud row (releasing the claim so the unit can be re-claimed). A resold or deleted unit leaves nothing behind.
- `factoryResetDevice(id)`: attaches the caller's session token, POSTs the ownership-checked `factoryReset` command (§7.2), then `removeDevice`. The confirmation dialog carries the data-loss disclaimer and the offline PRG fallback.
- `setIrrigation`, `runPump`: watering state and the simulated pump run (real pump control becomes a Losant command at the backend phase).
- Journal: `addJournalEntry` / `removeJournalEntry`, keyed by device id, entries `{id, date, photo, note}` with photos as downscaled data URLs.
- Gateways: `addGateway` / `removeGateway`, the Farm Kit bookkeeping list.
- Alarms: `addAlarmRule`, `addAlarmRules` (bulk, used by auto-set), `updateAlarmRule`, `removeAlarmRule`.
- `updateSettings`, `setTier`, plans sheet open/close.

The provider also derives the display identity: prefer `user_metadata.first_name + last_name` from signup, fall back to the email prefix, and expose `growerType` for the Settings badge.

9.6 store/helpers.js, the domain brain

- `METRICS`: per-metric metadata, label, unit, icon, color, slider min/max, and the default `good` / `warn` bands. `rainChance` exists as a pseudo-metric so rain can participate in the alarm system without being a device sensor.
- `PLANTS` (re-exported from `plants.js`, the searchable catalog with categories, scientific names, emoji, and per-plant range overrides). `rangesForDevice` layers the plant's ranges over the defaults, which is how "ideal" becomes species-relative everywhere at once.
- `statusOf(key, value, ranges)`: inside good = good; inside warn = warn; else critical. This single function colors the entire product.
- `metricConnected(key, reading)`: the disconnected-sensor detector, built from measured failure signatures: soil temperature below -100 (the DS18B20 -127 C sentinel), null values (DHT22 absent), humidity of zero or below (physically impossible from a working DHT22), and soil raw under 300 (a floating or unpowered probe; a real powered probe in air reads near its dry point of ~3600).
- `healthScore`: good=100, warn=66, critical=28, averaged over **connected** metrics only, so a missing probe neither drags down nor props up the score. The weights make a single critical metric visibly hurt (one critical among four healthy reads 82, not 95).
- `recommendations`: ordered advice. Missing-sensor fix-its first (each names the exact pin and component to check), then battery warnings (warn at 20 percent, critical at 10, suppressed while charging or on AC), then plant-care advice derived from the bands, then the all-good line only if nothing else fired.
- `powerInfo`: null `batteryPct` means AC (true for today's USB units); otherwise battery percent plus charging flag.
- Mock generators: `seedReading` and `nextReading` (bounded random walk; soil simulated in raw counts and converted exactly like the firmware does, same DRY/WET constants). `buildSeries` synthesizes hour/day/week/month chart data centered on the plant's ideal band for the in-app metric-detail sheets and the demo report; real reports now pull genuine long-range history from the cloud via `device-history` (§7.4).
- `activeAlerts(devices, rules, weather)`: evaluates enabled rules per device (or once against the forecast for `rainChance` rules) and returns the tripped set used by the banner, badge, and toasts. `alarmsFromPlant` generates the one-tap starter rules from the plant's good band.
- Display helpers: `convertTemp` / `displayValue` / `displayUnit` (F-to-C at the display layer only, storage stays Fahrenheit so units switching never mutates data), `trendOf` (last two history points: up, down, flat).
- `TRANSPORTS`: Wi-Fi and LoRaWAN. Ethernet was removed because current hardware has no port; honest options only.

9.7 store/tiers.js

Free ($0: 3 devices, core monitoring), Plus ($4.99/mo: 10 devices, weather rain gauge), Pro ($9.99/mo: unlimited, automated irrigation incl. rain delay, LoRaWAN positioning). Tiers are feature flags consumed across the app (`tier.weather`, `tier.irrigation`, `tier.deviceLimit`); demo mode runs on pro so prospects see everything; real accounts default to free with polished locked-state upgrade cards. Rain delay lives inside `tier.irrigation` (Pro) and is additionally gated at runtime on the node having a location, since it depends on the forecast.

9.8 Components

- **UI.jsx**, the primitive kit: icon maps (`MetricIcon`, `TransportIcon`), `PowerBadge` (AC plug or color-stepped battery icon, compact mode for tight cards), `Pills` (segmented control, supports disabled options with hint tooltips, used for the LoRaWAN-aware refresh rates), `Toggle`, `Slider`, `Stepper`, `Gauge` (the SVG moisture dial), `statusColor`.
- **Login.jsx**: sign-in and sign-up in one card. Signup adds first/last name (side by side), confirm-password with match and minimum-length validation, and the grower-type chip picker; submits metadata through `signup`. Enter submits; errors render inline; the demo button sits under the primary action.
- **Onboarding.jsx**: the empty-account welcome with Connect a device and Log out.
- **ClaimDeviceSheet.jsx**: the pairing flow described in chapter 8, including the Wi-Fi vs gateway chips whose helper text teaches the cadence trade-off (seconds vs minutes, battery life).
- **AddDeviceSheet.jsx**: demo-only simulated device creation.
- **GatewaySheet** (in DevicesView): name plus label code, with plug-into-router guidance.
- **WeatherCard.jsx**: maps Open-Meteo WMO weather codes to label/icon/color (`describe()`), shows temperature, condition, location, rain chance and high, plus contextual notes: a rain note suggesting skipped watering at 50 percent or more, a sun/heat note at high UV or heat. When `weather.needsLocation` it renders an "add this plant's location" prompt with an inline `SetLocationInline` instead, no fake city, no GPS request.
- **SetLocationInline.jsx**: a small self-contained control (city/ZIP input + button) that geocodes via `geocodePlace` and pins the result to the device with `updateDevice`. Reused by the weather card and the rain-delay gate so the customer can add a location without leaving what they're doing; an `onDone` callback lets the rain-delay gate immediately enable the feature once a location is saved.
- **Chart.jsx**: the recharts area chart, gradient fill, ideal-band `ReferenceArea`, max and average reference lines, animation off for live data.
- **GrowthJournal.jsx**: photo strip plus timeline sheet; photos are downscaled to thumbnails client-side (`fileToThumb`) before storage so localStorage stays manageable; capture attribute opens the camera on phones.
- **IrrigationCard.jsx**: manual/auto/schedule modes, target-moisture slider, pump duration stepper, Water now (simulated pump bumps moisture), the rain-pause banner with resume, and the **rain delay** card. Rain delay (skip auto-watering when rain is forecast) is gated on the node having a `geo`: the toggle is disabled without one and a warnbox explains that the feature needs *this specific plant's* home location for the forecast, with an inline `SetLocationInline` (`onDone` enables rain

delay the moment a location is saved). The auto-mode "skips when rain is forecast" line only appears when rain delay is actually on.

- **ExportSheet.jsx**: the report options sheet (period presets / custom from-to / "Everything", per-plant checkboxes when >1 device, per-sensor checkboxes). On generate it fetches each chosen plant's history (`device-history` for real units; a local mock series for demo plants) and calls `downloadAccountPdf` or `openAccountExport`. While working it swaps the form for a spinner + phase label ("Gathering your sensor data…" -> "Rendering your PDF…") and blocks close/backdrop so the in-flight render can't be orphaned or double-fired.
- **PlansSheet.jsx**: pricing cards from `TIERS`; in demo, buying sets the tier instantly and shows a thank-you (real billing is a roadmap item and the sheet says so).
- **PlantPicker.jsx**: search across name, scientific name, and category; results grouped by category; each row previews its ideal moisture band; picking re-ranges the whole app via `rangesForDevice`.

9.9 Views

- **LiveView**: the dashboard. `HERO` (three headline metrics) and `CHIPS` (four sensor chips) are key arrays mapped against `metricConnected`, connected values render colored by status, disconnected ones render gray dashes and "not connected" and are not tappable. The gauge/health card includes transport, power badge, and location. Weather and irrigation render or show their locked upgrade cards per tier. `RainPausePrompt` offers 24/48-hour watering pauses when rain probability is 50 percent or more. `MetricDetail` is the tap-through chart sheet. `DeviceWaiting` replaces the whole dashboard for a claimed device that has not reported yet.
- **HistoryView**: per-metric cards for HOUR/DAY/WEEK/MONTH with max/avg/min and the ideal band, unit-aware.
- **AlarmsView**: live alert banner and cards, the auto-set button (plant-derived starter rules), and rule cards with enable toggle, device scope dropdown, above/below pills, and a threshold slider bounded by the metric's min/max.
- **DevicesView**: grouped device cards (group headings plus "Other plants"; flat when no groups exist), the gateways section, both add flows, and the edit sheet: name, home location (re-geocoded on save), group (with datalist suggestions from existing groups), connection type with the honest switching note, and the danger zone, Remove and Factory reset, each behind a confirmation screen with explicit data-loss language.
- **SettingsView**: plan card, units, theme, the LoRaWAN-aware refresh-rate pills with the battery-drain note, notification channels (push/email/SMS with addresses), and the account card with name, grower badge, **Download report (PDF)** (opens `ExportSheet`), and sign-out. The old machine-readable JSON export link was removed, the branded report is the one export customers actually want.

9.10 Utilities

- **geocode.js**: Open-Meteo's free geocoder; returns `{lat, lon, label}` or null; the label ("Davie, Florida") doubles as the device's display location.

- **report.js**: the original single-plant snapshot report (header, plant identity, health stat, a sensor table of current values plus min/avg/max from in-memory history, weather, journal photos), opened in a new window for `window.print()`. Still used for the quick per-plant share.
- **accountExport.js**: the full account/plant **report** system, the bulk of the recent work. `buildReport()` assembles one branded HTML document from the selected plants and their fetched histories: a letterhead and account/period block, then per-plant **inline-SVG line charts** (one per selected sensor, ideal-band shading, min/avg/max, segmented at nulls so offline gaps and disconnected sensors show honestly), a chronological **activity timeline** (claimed date, first data, journal entries), and the device/gateway/alarm/preferences/journal summary tables. All chart CSS is scoped under `.gp-doc` / `.gpr-*` so the document can be rendered inside the live app page (for client-side PDF) without restyling the app, a real bug we hit when the report's `.brand` collided with the app's. Three delivery paths share that one document:
- `openAccountExport()` writes a standalone HTML doc to a new tab and auto-prints (sharpest text, paper-friendly).
- `downloadAccountPdf()` is the one-tap download: it **tries the server renderer first** (`render-pdf`, true vector PDF, §7.5), and on any failure or in demo mode **falls back** to `clientPdf()`.
- `clientPdf()` renders the report off-screen and rasterizes it with `html2pdf.js` (dynamically imported so it never weighs down normal app loads; PNG encoding and a 2–3x scale clamped to stay inside mobile-Safari's canvas limit). Slightly softer than the server's vector output but always available offline.

9.11 The design system (index.css)

- **Tokens**: `--bg #eef0f2`, `--card #fff`, ink scale (`--ink #2c3e50`, two muted steps), brand `--green #2ecc71` with dark accent `--green-d`, status colors, 22px card radius, layered soft shadows, `--maxw 480px` for the phone column.
- **Layout**: `.shell` is the phone-width column; at 860px it becomes a grid (`240px` sidebar plus content, `grid-template-areas`), the `.tabbar` converts from bottom bar to sidebar, and content widens. The app bar is sticky with backdrop blur.
- **Primitives**: card, badge (now `white-space: nowrap` after a wrapping bug), pills (with disabled state), chips, device cards (meta row wraps as a whole, reading column protected), sheets (bottom drawers with grab handle and slide-up animation, safe-area padded), overlay, toasts, skeletons, the warnbox (destructive confirmations), rolechips (signup picker), danger zone, fieldlabels.
- **Dark theme**: `:root[data-theme='dark']` overrides the token set; `auto` follows the OS.
- **Forms**: `input, select, textarea, button { font-family: inherit }`, browsers default form controls to system fonts with different metrics, which users perceive as "the typing looks shifted."
- **Safe areas**: `env(safe-area-inset-*)` padding on the app bar, tab bar, login, and sheets so the installed PWA clears the iPhone notch and home indicator.

9.12 PWA assets

`manifest.webmanifest` (name, standalone display, theme/background colors, 192 and 512 icons with maskable purpose) plus `apple-touch-icon.png` at 180px. The PNGs are rendered from the brand SVG

onto a white matte with the sharp library at build-tooling time (iOS renders transparency as black, hence the matte).

10. End-to-End Sequences

How the layers cooperate for each major operation, useful as integration documentation and as a debugging map.

First boot and provisioning. Power on, OLED "Starting up", boot-time PRG check, banner with pairing code, sensors begin, `connectWiFi()` finds no saved credentials, AP callback flips the OLED to join instructions, customer joins the hotspot, portal serves the branded config page (or 192.168.4.1 manually), credentials saved, association succeeds, OLED shows the pair code with "..", `connectLosant()` brings up MQTT, OLED flips to "ON", telemetry begins on the 3-second cadence.

Claim. App: Connect a device, transport choice, name, optional location, code. Store: registry SELECT by code (RLS-permitted read), reject if unknown; for a real account `insert` a `devices` row (unique `claim_code` enforces single ownership, a `23505` is surfaced as "already claimed"); geocode the place if given; add the device with `losantDeviceId` / `geo` / `claimedAt` and select it. UI shows DeviceWaiting until the first poll returns data, then the dashboard goes live.

Live tick. Interval fires, claimed devices fetch `device-state` in parallel, nulls and sentinels flow through untouched, readings append to history, trends/health/insights recompute, `metricConnected` decides which slots render as data versus "not connected."

Alarm trip. A reading crosses a rule threshold. In-app: `activeAlerts` includes it, the Alarms tab badge and banner update, the Shell's key-diff raises a toast. Out-of-app: the Losant workflow's conditional passes and the email/SMS node fires (subject to the one-per-minute email rate limit). Two independent layers, by design.

Factory reset. Edit device, danger zone, Factory reset, confirmation with the disclaimer, `factoryResetDevice`: attach the caller's Supabase session, POST device-command (POST + allowlist + write token + **ownership check** against the `devices` table), Losant delivers `factoryReset` over MQTT, firmware shows "Reset by owner", wipes Wi-Fi, reboots into the setup hotspot; meanwhile the app removes the device, deletes its cloud row (releasing the claim), and purges journal and device-scoped alarms. Offline unit: command is lost, PRG hold covers it, dialog says so.

Report export. Settings -> Download report -> `ExportSheet`: pick period / plants / sensors. On generate, fetch each plant's history (`device-history` per real unit, in parallel; mock series for demo), build the report via `accountExport.buildReport`. **Download PDF**: `downloadAccountPdf` POSTs the HTML to `render-pdf` (auth-checked headless-Chrome vector PDF) and saves the returned blob; on any failure or demo mode it falls back to `clientPdf` (html2pdf raster). **Print report**: `openAccountExport` opens the standalone HTML and auto-prints. A spinner with phase labels covers the wait and the sheet is close-locked until done.

11. Debugging Runbook

Every failure below was actually encountered during development. Symptoms are exact.

Symptom	Cause	Fix
Soil temperature -196.6 F	DS18B20 absent or, most often, missing/misplaced 4.7k pull-up (it is -127 C, the Dallas disconnected sentinel)	Bridge data and 3V3 with 4.7k; verify probe wiring; the app's insight names this exactly
Air temp/humidity nan or null	DHT22 absent or data wire loose	Check pin 5 and the module's pull-up
Soil moisture pinned at 100 percent, raw ~120, ignores water	Probe powered at 3.3V: NE555 oscillator dead below ~4V (LED still lights)	Power probe VCC from the 5V pin
Soil raw flat 0 on GPIO1	GPIO1 is the V3 battery-sense net	Use GPIO2 (firmware `SOIL_PIN 2`)
Floating ADC reads few-hundred noise, "looks alive"	Nothing driving the pin; ADC noise is normal	Judge by response to known voltage (jumper test: 3V3 reads ~4095, GND ~0), not by presence of a number
1-Wire scanner prints garbage addresses during wire taps	Contact bounce parsed as devices; real DS18B20 addresses start 0x28	Proves pin and code alive; fix the physical bus
Phone hangs "connecting..." to the setup hotspot	AP+STA channel hopping from background retries and modem sleep	`setWiFiAutoReconnect(false)`, `WiFi.setSleep(false)` (in firmware)
Captive portal never pops	Phone's portal detection suppressed (cellular data on, OS quirk)	Browse to 192.168.4.1; turning off cellular data helps
Portal vanished mid-setup	Old 180 s portal timeout rebooted the AP	Timeout is now 600 s
Board keeps joining old Wi-Fi instead of opening setup	Saved credentials still valid	Hold PRG 3 s (or PRG at power-on) to wipe
Upload dies: "chip stopped responding", exit status 2	921600 baud over a marginal cable	Upload speed 115200; PRG+RST for download mode; better cable
No serial port appears at all	CP210x driver missing	Install Silicon Labs CP210x VCP driver
`setPreloadWiFiScan` compile error	Setter absent from released WiFiManager (2.0.17)	Delete the line; the other two portal fixes carry the load

Symptom	Cause	Fix
Compile error: a file path inside the code	Accidental paste into the editor buffer	The error's caret points at it; delete the line
Cloud email alert "didn't fire"	One-per-minute Losant email rate limit, or spam folder	Space test readings a minute apart
404 "Application not found" from Losant REST	Token JWT subject is not the application id	App id comes from the dashboard URL, not from inside the token
Junk pairing code created a device (historic)	No validation	Registry lookup now rejects unknown codes
Claimed device shows fake-looking values (historic)	Devices seeded with simulated data	Claimed devices now start empty and honest
Welcome screen squeezed against the left edge on desktop (historic)	Onboarding rendered inside the shell grid, auto-placed into the 240px sidebar cell	Onboarding uses the full-page centered wrapper
Badge text wraps mid-word, card columns collide (historic)	Meta row could not wrap; badges could break internally	`white-space: nowrap` on badges, wrapping meta row, protected reading column
Typed text in fields looks subtly off	Form controls default to system fonts	`input/select/textarea/button { font-family: inherit }`
Plant report tab never opens	Popup blocked	Allow popups for growthpulsecloud.com once

12. Decision Log

Decision	Reasoning
Pairing code derived from the chip MAC and shown on the OLED	Unique per unit with zero provisioning cost; the customer reads it off the device; doubles as a serial number
Registry table validates every claim	Typos cannot create ghost devices; the table is the future factory database
Soil sensor on GPIO2	GPIO1 is wired to battery sense; measured flat 0
Soil probe powered at 5V	NE555 modules do not oscillate below ~4V; output stays ADC-safe
Sentinels pass through the pipeline untouched	The app converts them to honest "not connected" UI with fix-it hints; hiding them would mask hardware faults
Claimed devices start with no data	Never display invented numbers; trust is the product

Decision	Reasoning
Weather pinned to a per-device geocoded home	The forecast must follow the plant, not the traveling owner
Location is optional; never read browser/GPS; no default city	Customers who won't share location aren't blocked; we don't fabricate a place we can't justify
Rain delay is the only location-gated feature, and it asks at point of use	The one feature that genuinely needs a forecast is where we request the location, with the reason shown, and it names the node that still lacks one
Full report = cloud history + inline-SVG graphs + timeline, replacing JSON export	A branded report with graphs is what customers actually use; raw JSON was noise for ~99% of them
Report CSS scoped under `.gp-doc` / `.gpr-*`	The client-side renderer mounts the report inside the live app; unscoped class names collided with the app (centered logo bug)
Server-side PDF (headless Chrome) with a client raster fallback	Vector text is sharpest; the fallback guarantees the download always works (offline, demo, cold function)
`render-pdf` is auth-gated	Prevents abuse as an open HTML-to-PDF service while needing no Losant access
Cloud-authoritative ownership via a `devices` table with per-user RLS	Claims follow the account across browsers, enforce one owner per unit, and give `device-command` something to authorize against
History resolution auto-scaled to <=240 points	Reports stay readable and responses stay small whether the span is a day or a year
Health score excludes disconnected sensors; weights 100/66/28	A missing probe must not bias the score; one critical metric must visibly hurt
Two Losant API tokens (read-only and command)	Least privilege on the hot read path
All cloud tokens live in serverless functions	No credential ever ships in a browser bundle; `VITE_` prefix is the boundary
Command endpoint allowlists `factoryReset`	Bounds the blast radius of an as-yet-unauthenticated endpoint
Factory reset = cloud command + local purge + explicit disclaimer	A resale flow that physically readies the unit and leaves no data behind
Portal: 10-minute timeout, no auto-reconnect, no modem sleep	Fixes the observed slow-join and mid-setup restart failures
PRG 3-second hold replaces the PRG+RST timing dance	Customers reliably failed sub-second button choreography
Per-user localStorage namespacing plus provider remount on user change	Account isolation on shared browsers, and demo isolation for free

Decision	Reasoning
Demo as a fake user id rather than a flag sprinkled through code	One mechanism gives complete data separation
Ethernet removed from the UI	Current hardware has no port; honest options only
LoRaWAN-aware refresh options with battery notes	The UI teaches the trade-off where the user makes the choice
Reports via the browser print pipeline	Zero dependencies; Save-as-PDF, paper, and share for free
One-file firmware	Reviewable top to bottom; trivially flashable and handover-friendly at this scale

13. Known Limitations and Roadmap

1. **~~Per-unit cloud provisioning.~~ DONE (§14.2).** Boards self-provision at first boot: each posts its chip-derived pairing code plus a shared firmware token to `provision-device`, which creates (or reuses) a per-board Losant device and returns a device-scoped access key. One image, no per-board secrets, no flash-time station.
2. **~~Server-side device ownership.~~ DONE (§6.3, §7.2).** The `devices` table with per-user RLS is live: ownership is cloud-authoritative, syncs across browsers, enforces claim exclusivity, and `device-command` now verifies the caller owns the target before sending.
3. **Zero-typing auto-bind.** The portal carries a one-time account token; the unit POSTs it to a bind endpoint on first connect. Claim codes are the shipping-grade interim.
4. **~~LoRaWAN bring-up.~~ DONE (§14).** The combined firmware joins LoRaWAN through a ThinkNode G1 gateway on The Things Stack; uplinks route through the `lorawan-uplink` webhook into the same Losant device; transport switching is automated in both directions from the app; and gateways auto-register on the network. See the new §14 and the LoRaWAN System and Debugging Report.
5. **~~Battery telemetry.~~ DONE (§14.11).** GPIO37-controlled battery sense (polarity-corrected for this board), inferred charging state, smoothed percent, and an optional VBUS divider for true USB-versus-battery detection. `batteryPct` / `charging` flow through both transports.
6. **~~Durable history.~~ DONE (§7.4).** Reports now query Losant's time-series store through `device-history` for full-period graphs; the 60-reading in-memory ring remains only for the live trend/detail sheets.
7. **Web push notifications** for alarm trips, complementing cloud email/SMS.
8. **Billing.** Tiers are functional feature gates; Stripe integration replaces the demo "buy" path.
9. **Family sharing.** Multiple accounts per garden, now unblocked by item 2.
10. **Rain-delay actuation.** The rain-delay flag, location gating, and pause UX are in; wiring it to actually suppress real pump commands lands with the irrigation backend.

11. **Group-level report select.** Reports pick per node; a "whole group" shortcut is the next convenience once accounts run many nodes per zone.

---

14. Dual-Transport System: Self-Provisioning, Combined Firmware, and LoRaWAN

This section documents everything added after the June 5 revision: how a board gets its own cloud identity with no per-board secrets, the single image that runs either Wi-Fi or LoRaWAN, and the full LoRaWAN path through The Things Stack into the same Losant device the app already reads. The companion **GrowthPulse LoRaWAN System and Debugging Report** carries the chronological bug-by-bug record; this section is the reference architecture.

14.1 Two transports, two clouds, one device

GrowthPulse supports two ways for a node to reach the internet, and uses two managed clouds.

- **Losant** is the device cloud and the single source of truth the app reads. Every node, Wi-Fi or LoRaWAN, is represented as exactly one Losant device. This is the design choice that lets a node change transports and remain one plant in the app: both transports write to the same Losant device.
- **The Things Stack (TTS)** on The Things Network sandbox is the LoRaWAN network server. It owns gateways, OTAA join, and the LoRaWAN session, and forwards each uplink to a webhook. It stores no plant data; it is a bridge from the radio world into Losant.

A Wi-Fi node publishes to Losant directly over MQTT. A LoRaWAN node reaches a gateway, which reaches TTS, which calls the `lorawan-uplink` webhook, which writes to Losant. Different path in, same destination.

14.2 Wi-Fi self-provisioning (provision-device, device_registry)

The "mail a customer a board" requirement means one image with no per-board secrets. A board earns its identity at first boot.

1. The board derives a **pairing code** from its chip hardware id (the high bits of the efuse MAC). This is deterministic, so the same physical board always produces the same code, even after a factory reset.
2. On first boot in Wi-Fi mode, after connecting to the network, the board POSTs `{ code, token }` to `provision-device.js`, where `token` is the shared firmware provisioning token (the same value as the Netlify `PROVISION_TOKEN`).
3. The function looks the code up in the **device_registry** table. If the code is new, it creates a Losant device with the full attribute schema, registers the code-to-device mapping, and returns a fresh

device-scoped access key. If the code already exists (a re-provision after a reset), it returns the same device, so a board never spawns a duplicate.
4. The board saves the device id, access key, and secret to NVS and connects to Losant over MQTT.

As of v2.22 the function also re-applies the full attribute schema on every call (a PATCH), so a device created before a newer attribute existed gains it on the next provision. This is what fixed the LoRaWAN state rejection in §14.14.

The shared token is the only secret in the image, and the repository copy carries a placeholder; the real value is filled in only in a private build and is also set as the Netlify `PROVISION_TOKEN` so the function can authenticate the board.

14.3 The combined firmware (GP_Combined): boot path and OLED

`firmware/GP_Combined/GP_Combined.ino`, version 4.2, is the production image. A `netmode` flag in the NVS namespace `gp` selects the boot path: `wifi` (default) or `lorawan`.

- **Wi-Fi path**: try saved Wi-Fi; if none or it fails, open the phone setup hotspot; self-provision if there is no saved Losant identity; then stream telemetry to Losant over MQTT every few seconds. Telemetry stamps `transport: "wifi"`, `wifiRssi`, `batteryPct`, and `charging`.
- **LoRaWAN path**: bring up the SX1262, load OTAA keys from NVS, join through any gateway, and uplink a 9-byte payload every interval.

The OLED cycles through three pages (about 5 seconds each): the pairing code with online status, the live connection page (link type, Wi-Fi dBm or LoRa RSSI and SNR, and battery or power), and the live sensor readings. The branded setup portal markup is shared with the Wi-Fi-only firmware.

Mode is set three ways and each writes NVS then reboots: the portal's Connection field, a Losant `setMode` command (Wi-Fi units), or a LoRaWAN downlink (LoRaWAN units). The image overflows the default app partition, so it is built with a larger-app or "Huge App" partition scheme.

14.4 LoRaWAN on the node: SX1262, OTAA, payload, downlinks

The SX1262 pins are the Heltec V3 schematic values: NSS 8, DIO1 14, RST 12, BUSY 13, SCK 9, MISO 11, MOSI 10. Bring-up uses RadioLib's default 1.6V TCXO and `setDio2AsRfSwitch(true)`; US915 uses `setDutyCycle(false)` (fair-use airtime, not a duty cycle) and sub-band 2 (FSB2).

OTAA is a two-step sequence in RadioLib, and getting this wrong cost a debugging cycle (§14.14, issue 10): `beginOTAA(joinEUI, devEUI, nwkKey, appKey)` only sets the keys and returns success without joining; `activateOTAA()` performs the actual join and returns a new-session or restored-session result. The firmware restores the saved join nonces, calls `beginOTAA`, then loops on `activateOTAA` with a 10-second backoff until it joins, persisting nonces between attempts so DevNonce keeps incrementing.

Uplinks use the 8-argument `sendReceive(payload, len, fport, downBuf, &downLen, false, nullptr, &evDown)` form so a downlink can be captured in the same call. The 9-byte payload:

```
[0..1] soilTemperatureF * 10, int16 big-endian  (0x8000 = sensor absent)
[2..3] airTemperatureF  * 10, int16 big-endian
[4]    airHumidity percent,    uint8
[5..6] soilRaw 0..4095,        uint16 big-endian
[7]    soilMoisturePercent,    uint8
[8]    batteryPct 0..100,      uint8   (0xFF = unknown)
```

A one-byte 0x00 downlink means "switch back to Wi-Fi"; the firmware writes `netmode = wifi` and reboots. The uplink interval (`LORA_UPLINK_MS`) is 60 seconds for bench and demo responsiveness and is documented to raise toward 15 minutes for The Things Network fair-use in production (§14.14, issue 22).

14.5 The Things Stack topology and the four-call device API

A critical, non-obvious fact about The Things Network: the **Identity Server** (which registers devices and gateways) is centralized on the **eu1** cluster, while a device's **Network, Join, and Application servers** run on the **operating cluster** (nam1 for North America). Creating a device through the API therefore touches two hosts and four endpoints:

1. **Identity Server** (eu1): `POST /api/v3/applications/{app}/devices` creates the registration, with the network, application, and join server addresses set to the operating cluster.
2. **Join Server** (nam1): `PUT /api/v3/js/applications/{app}/devices/{id}` sets the root key (AppKey). This is the only call that carries the key.
3. **Network Server** (nam1): `PUT /api/v3/ns/...` sets the frequency plan, MAC and PHY versions, `supports_join` , and `resets_join_nonces` .
4. **Application Server** (nam1): `PUT /api/v3/as/...` registers the device.

Each server only accepts the field-mask paths it owns; sending one a path that belongs to another is rejected as a forbidden path (§14.14, issue 12). The four-call split and the eu1-versus-nam1 host split are both encoded in `provision-lorawan.js` .

14.6 provision-lorawan.js: idempotent provisioning

This function is the LoRaWAN twin of self-provisioning, called when the app switches a node to LoRaWAN. It is idempotent: **one TTS device per board**, which is the most important correctness property of the whole LoRaWAN system (§14.14, issues 14 and 8 of the report).

1. Validate the signed-in user. Read the board's Losant id from the request.
2. Look the board up in **lorawan_devices** by its Losant id. If a row with a stored AppKey exists, reuse that TTS device: clear its downlink queue, set `resets_join_nonces` , and re-push the stored keys to the board through a Losant `provisionLoRa` command. Return.
3. If there is no row, generate a DevEUI and AppKey, create the TTS device through the four-call API, store the route (TTS device id, DevEUI, AppKey, Losant id, user id), and push the keys.

Storing the AppKey in `lorawan_devices` is what makes reuse possible: the same keys can be re-pushed without minting a new device. A non-2xx Supabase response is checked explicitly, because a swallowed route-store failure leaves the webhook unable to map the board (§14.14, issue 13).

14.7 lorawan-uplink.js: the webhook decode-and-route

The webhook TTS calls on every uplink. It is deliberately self-sufficient.

1. Verify the shared `X-Webhook-Token` header against `LORAWAN_WEBHOOK_TOKEN`.
2. Take the uplink's `decoded_payload` if a TTS formatter set one; otherwise **decode the 9 raw bytes itself** from `frm_payload`. This removes any dependency on a per-device TTS payload formatter, which auto-provisioned devices do not have (§14.14, issue 13).
3. Resolve the target Losant device from `lorawan_devices` by the uplink's TTS device id (with a static env-map and a default as fallbacks).
4. Build the same state shape the Wi-Fi path sends, tag it with `transport: "lorawan"`, `loraRssi`, and `loraSnr`, omit any null field (Losant rejects a null for a typed Number attribute), and POST it to the Losant device.

As of v2.22 it logs the exact outcome of each delivery (resolved device and a 200/401/404/422/502 with the rejection detail), because a silently failing webhook gets auto-deactivated by TTS and looks like it is not firing at all.

14.8 lorawan-switch-wifi.js and register-gateway.js

lorawan-switch-wifi.js brings a LoRaWAN node back to Wi-Fi. A LoRaWAN node is off Losant's MQTT, so a Losant command cannot reach it; the only path down is a LoRaWAN downlink, delivered during the node's next uplink window. The function looks up the board's TTS device by its Losant id and enqueues a one-byte 0x00 downlink on fPort 2 through the Application Server's `down/replace`. The node's firmware sees the 0x00 and reboots into Wi-Fi.

register-gateway.js registers a customer's gateway on TTS under the GrowthPulse network so the customer never touches The Things Stack. The gateway registration is an Identity Server call (eu1), with the gateway server address pointing at the operating cluster. The gateway hardware must still be pre-configured before shipping to forward to the GrowthPulse TTS server on US915 FSB2; the app cannot reach inside the gateway to set that.

14.9 Supabase tables: lorawan_devices (v2) and gateways

lorawan_devices (migration `docs/growthpulse-lorawan-routes-schema-v2.sql`) maps a TTS end-device to the Losant device its uplinks land on:

```
tts_device_id     text primary key
dev_eui           text not null
app_key           text                        -- OTAA root key, re-pushed on reuse
```

```
losant_device_id  text not null              -- UNIQUE: one TTS device per board
user_id           uuid references auth.users
created_at        timestamptz default now()
```

The unique index on `losant_device_id` enforces one TTS device per board. The v2 migration adds the `app_key` column and the unique index and clears v1 orphan rows.

gateways (migration `docs/growthpulse-gateways-schema.sql`) stores LoRaWAN gateways on the account so they persist across logins.

Both tables have Row Level Security on with no client policies; only the serverless functions, using the service role, read or write them.

14.10 Transport switching, both directions, and the deadlock

Wi-Fi to LoRaWAN works because the board is reachable on Losant while on Wi-Fi: the backend pushes keys through a Losant command, the board saves them and reboots into LoRaWAN, and joins.

LoRaWAN to Wi-Fi is the hard direction. The board is off Losant, so the switch is a queued LoRaWAN downlink that lands on the next uplink. The app gates the switch on the **saved** transport, not the live card, so a momentarily wrong card cannot make the button inert (§14.14, issue 15).

The deadlock to be aware of: provisioning or switching to LoRaWAN requires the board to be on Wi-Fi (to receive the Losant command). A board stuck on LoRaWAN cannot be reached that way. The escape hatch is the PRG button's 3-second graceful return to Wi-Fi (§14.12), which keeps saved credentials.

14.11 Battery, charging, and the VBUS mod

The Heltec V3 battery sense is read through a divider gated by GPIO37. On this board the control polarity is inverted from the documented reference, so the firmware drives it to the opposite level and averages 16 ADC samples. There is no charge-status pin, so charging is inferred from the voltage trend and the top-of-charge voltage. The reported percent is smoothed to creep at most 1% per reading, so a charger's instantaneous voltage inflation does not show as a jump.

With no battery installed on USB, the charge chip pins the sense line near a full pack's voltage and the board has no battery-detect pin, so this cannot be fully resolved in software; the firmware reports AC when the line is pinned at the top and no longer rising. The honest hardware fix is an optional 100k/100k divider from the board's 5V pin to ground with the midpoint on GPIO7 (`USE_VBUS_SENSE` set to 1): about 2.5V on USB, 0V on battery. Default off.

14.12 The PRG button state machine and command grace

The PRG button (GPIO0) is tiered, acting on release:

- **Single tap**: wake the OLED or wake from sleep.

- **Double tap** (two short presses): enter deep sleep (microamp draw, screen off), the soft power-off. A single press or RESET wakes it.
- **Hold 3 seconds**: graceful return to Wi-Fi, keeping saved Wi-Fi credentials, for recovering a stuck LoRaWAN unit.
- **Hold 10 seconds**: full factory reset (wipe Wi-Fi, reopen setup). The OLED shows which action is pending while the button is held.

A separate guard protects against stale cloud commands: the firmware ignores `provisionLoRa`, `factoryReset`, and `setMode` for the first 12 seconds after connecting to Losant, because a command delivered the instant a device connects is a stale replay, not a user action (§14.14, issue 17).

14.13 Environment variables (LoRaWAN additions)

In addition to the variables in §7.3, the LoRaWAN path uses:

Variable	Used by	Purpose
`TTS_API_KEY`	provision-lorawan, lorawan-switch-wifi, register-gateway	TTS API key with device and gateway write rights
`TTS_APP_ID`	provision-lorawan, lorawan-switch-wifi	the TTS application id (growthpulse)
`TTS_CLUSTER`	same	operating cluster host, default nam1.cloud.thethings.network
`TTS_IS_HOST`	provision-lorawan, register-gateway	Identity Server host, default eu1.cloud.thethings.network
`TTS_USER_ID`	register-gateway	the TTS account that owns registered gateways
`TTS_FREQ_PLAN`	provision-lorawan, register-gateway	default US_902_928_FSB_2
`LORAWAN_WEBHOOK_TOKEN`	lorawan-uplink	shared secret the TTS webhook sends as X-Webhook-Token
`PROVISION_TOKEN`	provision-device	shared firmware token authenticating a board's self-provision
`LOSANT_PROVISION_API_TOKEN`	provision-device	Losant token with device-create rights

14.14 Failure modes and how to read them

The full chronological journal is in the LoRaWAN System and Debugging Report. The highest-value reference points:

- **Join fails with -1116 (no join accept)**: either a sub-band mismatch (gateway on FSB1, TTS on FSB2) or DevNonce catch-up after a reflash (set `resets_join_nonces`).
- **First uplink fails with -1101 (session not active)**: `activateOTAA` was never called; `beginOTAA` only sets keys.
- **Downlink rejected with `no_device_session`**: it is aimed at a TTS device the node never joined, an orphan from non-idempotent provisioning.
- **App card stuck on Wi-Fi and the TTS webhook auto-deactivated**: Losant is rejecting the LoRaWAN state because the device lacks the `loraRssi` / `loraSnr` attributes; provision-device's schema sync fixes it on the next check-in.
- **Node bounces between Wi-Fi and LoRaWAN**: a stale `provisionLoRa` or `factoryReset` command on reconnect (the 12-second grace) or a stale 0x00 downlink in the queue (the backend queue-clear).
- **A TTS webhook showing "Healthy" with empty function logs** has likely been auto-deactivated for repeated failures; read its status banner.

---

15. Appendix

15.1 Links

Resource	URL
Live app	https://growthpulsecloud.com
App repository	https://github.com/BryanPuckettGH/growthpulse-app
Netlify	https://www.netlify.com
Supabase	https://supabase.com
Losant	https://www.losant.com
Open-Meteo (forecast + geocoding)	https://open-meteo.com
Heltec V3 docs	https://wiki.heltec.org
ESP32 Arduino boards index	https://espressif.github.io/arduino-esp32/package_esp32_index.json
CP210x USB driver	https://www.silabs.com/developer-tools/usb-to-uart-bridge-vcp-drivers
Arduino IDE	https://www.arduino.cc/en/software

15.2 Project file inventory

Path	Contents
`growthpulse-app/`	The web app and serverless functions (this repo deploys the product)
Firmware	
`firmware/GP_Combined/GP_Combined.ino`	Production image: one image, Wi-Fi or LoRaWAN by NVS flag (v4.2)
`firmware/GP_Node/GP_Node.ino`	Wi-Fi-only self-provisioning image (reference)
`firmware/GP_LoRaWAN/GP_LoRaWAN.ino`	Standalone LoRaWAN sketch (the proven OTAA reference)
`firmware/GP_Provisioning/GP_Provisioning.ino`	Original captive-portal firmware (reference)
Serverless functions	
`netlify/functions/provision-device.js`	Wi-Fi self-provision: create/reuse Losant device, sync attributes
`netlify/functions/provision-lorawan.js`	Idempotent LoRaWAN provisioning (one TTS device per board)
`netlify/functions/lorawan-uplink.js`	TTS uplink webhook: decode bytes, route to Losant
`netlify/functions/lorawan-switch-wifi.js`	Enqueue the 0x00 downlink to return a node to Wi-Fi
`netlify/functions/register-gateway.js`	Auto-register a customer gateway on TTS
`netlify/functions/device-state.js`	Read path the app polls (transport + signal aware)
`netlify/functions/device-command.js`	Owner-verified cloud command (factory reset, etc.)
`netlify/functions/device-history.js`	Time-series history for report graphs
`netlify/functions/render-pdf.js`	Headless-Chrome vector PDF renderer
Supabase schemas	
`docs/growthpulse-supabase-schema.sql`	Pairing-code registry
`docs/growthpulse-devices-schema.sql`	Ownership `devices` table + RLS
`docs/growthpulse-lorawan-routes-schema-v2.sql`	`lorawan_devices` route table (current)
`docs/growthpulse-gateways-schema.sql`	Account-saved gateways

Path	Contents
Documentation	
`docs/GrowthPulse User Manual.(md/pdf)`	Customer-facing manual
`docs/GrowthPulse Engineering Manual.(md/pdf)`	This document
`docs/GrowthPulse LoRaWAN System and Debugging Report.md`	Dual-transport architecture + full bug journal
`docs/GrowthPulse LoRaWAN Bring-Up Guide.(md/pdf)`	One-time by-hand gateway + TTS setup
`docs/GrowthPulse LoRaWAN Auto-Provision Setup.md`	Env vars + flow for automated provisioning
`docs/GrowthPulse Gateway Auto-Register Setup.md`	Env vars + flow for gateway auto-registration
`docs/GrowthPulse LoRaWAN Cleanup and Reset.md`	Operational reset runbook
`docs/GrowthPulse Self-Provisioning Setup.(md/pdf)`	Wi-Fi self-provision setup
`docs/GrowthPulse Wiring Guide.md`, `docs/GrowthPulse Wiring Diagram.pdf`	Bench wiring references
`docs/GrowthPulse Product Roadmap.md`, `docs/GrowthPulse Deployment Model.md`	Product strategy docs
`src/utils/accountExport.js`	Full report builder + three delivery paths
`README.md` (repo)	Developer onboarding

15.3 Identifier reference (non-secret)

Identifier	Value
Demo unit pairing code	4A7AC
Demo unit hotspot	GrowthPulse-04A7AC
Demo Losant device id	6a1fb486df527c8bf8d3324b
Serial baud	115200
Portal fallback address	192.168.4.1

Access keys, access secrets, API tokens, and the Supabase anon key are intentionally absent from this document. They live in the firmware file, the gitignored `.env`, and Netlify's environment store.

GrowthPulse Engineering, internal and confidential. Keep credentials out of documents and repositories.